

Professional Industrial Yield Optimization Platform

HTML + Flask Architecture

This version is designed to look more enterprise-grade and production-like compared to Streamlit.

Recommended for:

- Bootcamp demos
- Portfolio projects
- Enterprise AI showcase
- Industrial analytics platform simulation
- Carrier Global presentation

Why HTML + Flask Is Better For Professional UI

Advantages

Better UI/UX

You get:

- Full-screen dashboard
- Sidebar navigation
- Professional cards
- Animated charts
- Responsive design
- Enterprise styling
- Better branding

Better Architecture

Closer to real-world industrial systems:

Frontend (HTML/CSS/JS)

↓

Flask API Backend

↓

ML Services



Database

Easier Future Scaling

You can later add:

- Login authentication
- APIs
- Real databases
- Real-time streaming
- Docker
- Cloud deployment
- Kafka
- Digital twin simulation

Recommended Final Tech Stack

Frontend

- HTML5
- CSS3
- Bootstrap 5
- JavaScript
- Plotly.js
- Font Awesome

Backend

- Flask
- Flask REST API
- Python

ML

- scikit-learn
 - pandas
 - numpy
-

Database

Start with:

- CSV files

Then upgrade to:

- SQLite
 - PostgreSQL
-

FINAL RECOMMENDED ARCHITECTURE

```
yield-optimization-platform/
|
├─ app.py
├─ requirements.txt
├─ README.md
├─ .gitignore
|
├─ data/
|   ├── manufacturing_data.csv
|   └── processed_data.csv
|
├─ models/
|   ├── train_model.py
|   ├── predict.py
|   ├── recommendation_engine.py
|   └── yield_model.pkl
|
├─ simulator/
|   └── generate_data.py
|
├─ static/
|   ├── css/
|   |   ├── style.css
|   |   └── dashboard.css
|   |
|   └── js/
|       ├── dashboard.js
|       ├── charts.js
|       └── animations.js
|
```

```
| | | images/
| | | | logo.png
| | | | factory.jpg
| | | | background.jpg
| | | icons/
| |
| | templates/
| | | base.html
| | | dashboard.html
| | | monitoring.html
| | | benchmarking.html
| | | recommendations.html
| | | ai_insights.html
| | | semantic_layer.html
| |
| | routes/
| | | dashboard_routes.py
| | | api_routes.py
| | | recommendation_routes.py
| |
| | utils/
| | | preprocessing.py
| | | helpers.py
| | | chart_helpers.py
| |
| | database/
| | | db_config.py
| | | schema.sql
| |
| | notebooks/
| | | experimentation.ipynb
```

UI DESIGN CONCEPT

Theme

Smart Factory + Industrial AI Command Center

Visual Style

Inspired by:

- Manufacturing execution systems
 - Siemens dashboards
 - Honeywell industrial systems
 - GE Digital dashboards
 - Power BI executive dashboards
 - Modern SaaS analytics platforms
-

Color Palette

Color	Use
#0B132B	Background
#1C2541	Sidebar
#3A506B	Cards
#5BC0BE	Accent
#00D9FF	Highlights
#FFFFFF	Text

MAIN DASHBOARD FEATURES

1. Executive Dashboard

KPI Cards

Show:

- Average Yield
 - Scrap Rate
 - OEE
 - Quality Score
 - Energy Consumption
 - AI Adoption Rate
-

Charts

- Yield trend
 - Scrap trend
 - Downtime trend
 - Production throughput
-

2. Process Monitoring

Live-like sensor charts:

- Temperature
- Pressure
- Humidity
- Machine speed
- Vibration

Add:

- Threshold alerts
 - Status indicators
 - Anomaly flags
-

3. AI Recommendation Center

Professional recommendation cards:

AI Recommendation

Plant: Plant A

Line: Line 2

Issue:

Humidity above threshold.

Recommended Action:

Reduce humidity below 55%.

Expected Yield Improvement:

+4.1%

Confidence Score:
92%

4. Semantic KPI Governance

This section makes your project look enterprise-grade.

Show:

KPI	Definition	Owner	Source
Yield %	Good units / total units	Manufacturing	Historian
Scrap Rate	Scrap units / total	Production	MES
OEE	Availability × Performance × Quality	Operations	ERP

5. Cross Plant Benchmarking

Compare:

- Plant performance
- Yield efficiency
- Scrap
- Energy
- Downtime

Charts:

- Radar chart
- Bar chart
- Heatmap
- Ranking table

6. AI Insights Engine

Generate business insights:

Examples:

- High humidity strongly correlates with yield loss.
- Plant B performs better during morning shift.

- Vibration spikes increase scrap probability.
- Line 3 has highest downtime.

DATABASE DESIGN

Main Table — manufacturing_data

Column	Type
id	INTEGER
timestamp	DATETIME
plant_id	VARCHAR
production_line	VARCHAR
batch_id	VARCHAR
temperature	FLOAT
pressure	FLOAT
humidity	FLOAT
machine_speed	FLOAT
vibration	FLOAT
raw_material_quality	FLOAT
energy_consumption	FLOAT
operator_shift	VARCHAR
yield_percentage	FLOAT
scrap_rate	FLOAT
quality_score	FLOAT
downtime_minutes	FLOAT
predicted_yield	FLOAT
adoption_flag	INTEGER

PROFESSIONAL PAGE LAYOUT

Sidebar

Contains:

- Logo
 - Navigation menu
 - Plant filters
 - Date filters
 - Theme toggle
-

Main Area

Contains:

- KPI cards
 - Interactive charts
 - AI recommendation widgets
 - Benchmarking tables
 - Insights panels
-

HTML TEMPLATE DESIGN

base.html

Main reusable layout.

Contains:

- Sidebar
- Navbar
- Footer
- Bootstrap imports
- JS imports

All pages inherit from this.

dashboard.html

Main executive overview.

Contains:

- KPI cards
 - Trend charts
 - Benchmarking section
 - Production summary
-

monitoring.html

Industrial process monitoring screen.

Contains:

- Sensor trends
 - Machine status
 - Alert panels
 - Heatmaps
-

recommendations.html

AI recommendation center.

Contains:

- Recommendation cards
 - Predicted improvement
 - Confidence score
 - Operator adoption tracking
-

semantic_layer.html

Enterprise semantic governance layer.

Contains:

- KPI catalog
 - Business glossary
 - Data lineage simulation
 - KPI ownership
-

FRONTEND COMPONENTS

KPI Card Example

```
<div class="kpi-card">
  <h3>Average Yield</h3>
  <h1>92.4%</h1>
</div>
```

Recommendation Card Example

```
<div class="recommendation-card">
  <h4>AI Recommendation</h4>
  <p>Reduce humidity below 55%</p>
  <span>Expected Gain: +4.2%</span>
</div>
```

REQUIRED PYTHON FILES

generate_data.py

Creates synthetic industrial manufacturing data.

train_model.py

Trains:

- Yield prediction model
- Scrap prediction model

predict.py

Makes predictions from incoming process parameters.

recommendation_engine.py

Business logic for recommendations.

Example:

```
if humidity > 60:  
    recommendation = "Reduce humidity below 55%"
```

PROFESSIONAL FEATURES TO ADD

Highly Recommended

1. Animated Counters

KPI values animate.

2. Gauge Charts

For:

- Yield
 - OEE
 - Quality score
-

3. Dark Mode Dashboard

Looks modern and industrial.

4. Notification Panel

Example:

```
ALERT:
Plant B humidity exceeded threshold.
```

5. AI Assistant Section

Example:

```
AI Insight:
Humidity and vibration are major contributors to yield loss.
```

DEPLOYMENT OPTIONS

Free Deployment

Frontend + Flask

Use:

- Render
 - Railway
 - PythonAnywhere
-

REQUIREMENTS.TXT

```
flask
pandas
numpy
plotly
scikit-learn
faker
joblib
unicorn
```

STARTER FLASK APP

app.py

```
from flask import Flask, render_template
import pandas as pd

app = Flask(__name__)

@app.route('/')
def dashboard():

    df = pd.read_csv('data/manufacturing_data.csv')

    avg_yield = round(df['yield_percentage'].mean(), 2)
    avg_scrap = round(df['scrap_rate'].mean(), 2)

    return render_template(
        'dashboard.html',
        avg_yield=avg_yield,
        avg_scrap=avg_scrap
    )

if __name__ == '__main__':
    app.run(debug=True)
```

STARTER DASHBOARD HTML

templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Yield Optimization Platform</title>

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
bootstrap.min.css"
        rel="stylesheet"
    >

    <link
```

```

        rel="stylesheet"
        href="{{ url_for('static', filename='css/style.css') }}"
    >
</head>
<body>

<div class="container-fluid">

    <div class="row">

        <div class="col-md-2 sidebar">
            <h2>🏭 Smart Factory AI</h2>
        </div>

        <div class="col-md-10 main-content">

            <h1>Executive Dashboard</h1>

            <div class="row mt-4">

                <div class="col-md-3">
                    <div class="kpi-card">
                        <h5>Average Yield</h5>
                        <h1>{{ avg_yield }}%</h1>
                    </div>
                </div>

                <div class="col-md-3">
                    <div class="kpi-card">
                        <h5>Scrap Rate</h5>
                        <h1>{{ avg_scrap }}%</h1>
                    </div>
                </div>

            </div>

        </div>

    </div>

</div>

</body>
</html>

```

STARTER CSS

static/css/style.css

```
body {
  background-color: #0B132B;
  color: white;
  font-family: Arial, sans-serif;
}

.sidebar {
  background-color: #1C2541;
  min-height: 100vh;
  padding: 20px;
}

.main-content {
  padding: 30px;
}

.kpi-card {
  background-color: #1C2541;
  padding: 20px;
  border-radius: 15px;
  text-align: center;
  box-shadow: 0 4px 12px rgba(0,0,0,0.3);
}

.kpi-card h1 {
  color: #00D9FF;
}
```

HOW TO RUN

Install

```
pip install -r requirements.txt
```

Generate Data

```
python simulator/generate_data.py
```

Run Flask App

```
python app.py
```

FINAL RECOMMENDATION

For your bootcamp:

BEST OPTION:

Flask + HTML + Bootstrap + Plotly

Because:

- Looks enterprise-grade
 - More visually impressive
 - Closer to real industrial software
 - Strong portfolio project
 - Better storytelling during demo
 - Easier to present as production-ready
-

WHAT WE SHOULD BUILD NEXT

Next step:

1. Full professional dashboard UI
2. Sidebar navigation
3. Interactive Plotly charts
4. Advanced CSS styling
5. Yield prediction model
6. AI recommendation engine
7. Semantic layer page
8. Cross-plant benchmarking
9. Industrial animations

10. Real database integration